

RETAILMART V3 • SQL

DDL & DML

WhatsApp Announcement

DDL & DML: Build Your First Tables, Then Fill Them!

Yesterday you explored RetailMart V3 - 55 tables, millions of rows. You looked at the data. Today you go from spectator to architect AND builder: first you BUILD tables (DDL), then you put DATA into them (DML).

Today's Focus:

- CREATE DATABASE, CREATE SCHEMA, CREATE TABLE - building structure from scratch
- Every column is TEXT today - we focus 100% on STRUCTURE, not types (types come tomorrow)
- DROP, IF NOT EXISTS, naming conventions
- INSERT - add rows. UPDATE - change rows. DELETE - remove rows. TRUNCATE - empty a table

Important: ALL building and data-entry today happens in YOUR practice database (accio_NN), NOT in RetailMart. You never change RetailMart's tables - you observe its design and build your own.

Course Portal: <https://starlordali4444.github.io/postgresql/>

Today you write your FIRST real SQL and put your FIRST rows in. Let's go!

- Siraj Sir

#Day3 #DDL #DML #PostgreSQL

TODAY'S AGENDA (2 Hours)

Section	Time	Topics
Quick Recap	5 mins	5 SQL categories, Database hierarchy, pgAdmin connection check
Topic 1: DDL - Building Structures	35 mins	CREATE DATABASE / SCHEMA / TABLE, Why every column is TEXT today, DROP, IF NOT EXISTS, Naming conventions
Practice Block 1	20 mins	4 problems: Build tables from scratch (all TEXT columns)
Topic 2: DML - Putting Data In	40 mins	INSERT (single, multi-row, RETURNING), UPDATE (always WHERE!), DELETE, TRUNCATE, DROP vs TRUNCATE vs DELETE
Practice Block 2	25 mins	4 problems: INSERT / UPDATE / DELETE / TRUNCATE on your tables
Wrap-up	15 mins	Summary, self-check, homework, Data Types & Constraints preview

5-Minute Memory Check

From SQL Intro:

- 5 SQL Categories: DDL, DML, DQL, DCL, TCL
- Database Hierarchy: Server -> Database -> Schema -> Table -> Row -> Cell
- SQL is DECLARATIVE - you say WHAT, engine decides HOW

From Setup & Onboarding:

- PostgreSQL = server process. pgAdmin = GUI client. psql = terminal client.
- RetailMart V3 loaded - 16 schemas, 55 tables
- You ran `SELECT count(*)` and `SELECT * ... LIMIT 5`
- You browsed schemas: customers, sales, products, stores, finance...

Today's Big Question:

'Those 55 tables did not appear by magic.
Someone wrote `CREATE TABLE` 55 times (DDL).
Someone wrote millions of `INSERTs` to fill them (DML).
Today, YOU do both - on your OWN tables.'

The Architect, the Builder, and the Resident (1/2)

Three people work on a building:

The Architect (DDL):

Draws the blueprint. How many floors? How many rooms? What is each room called? The architect defines STRUCTURE - she never moves furniture in.

The Builder (DML):

Follows the blueprint and puts things INSIDE - moves furniture in (INSERT), rearranges it (UPDATE), removes it (DELETE). The builder cannot add a new room - that needs the architect.

The Architect, the Builder, and the Resident (2/2)

The Resident (DQL):

Lives in the house and looks around (SELECT). Changes nothing. You meet the Resident starting tomorrow-ish (Filtering & Sorting).

Today you are BOTH the Architect (Topic 1: DDL) and the Builder (Topic 2: DML).

[Mapping to SQL]

Architect draws blueprint	-> CREATE TABLE	(define structure)	[Topic 1]
Architect modifies blueprint	-> ALTER TABLE	(change structure)	[tomorrow]
Architect demolishes a room	-> DROP TABLE	(destroy structure)	[Topic 1]
Builder brings furniture	-> INSERT	(add data)	[Topic 2]
Builder rearranges furniture	-> UPDATE	(change data)	[Topic 2]
Builder removes furniture	-> DELETE	(remove data)	[Topic 2]
Resident looks around	-> SELECT	(read data)	[later]

Today's Big Idea: Every Column Is TEXT

A table is a STRUCTURE: named columns, in order, grouped into a schema. Every column must have a TYPE - and today we give EVERY column the simplest, most forgiving type: TEXT. TEXT accepts any characters - letters, digits, dates typed as text, anything.

Why TEXT for everything today?

- So you focus 100% on STRUCTURE - what tables exist, what they are named, what columns they hold, how schemas organize them.
- Choosing the RIGHT type for each column (INT, NUMERIC, DATE, BOOLEAN...) is a whole skill - that is TOMORROW (Data Types).
- Tomorrow you will ALTER these TEXT columns into their proper types - motivated by the real data you INSERT today.

So today's CREATE TABLE is beautifully simple: 'column_name TEXT' for every column. No types to agonize over, no rules yet. Just structure.

CREATE DATABASE - The Empty Plot of Land

A database is the top-level container. Everything lives inside it - schemas, tables, views, indexes, functions.

[Syntax & Examples]

-- Basic syntax

```
CREATE DATABASE database_name;
```

-- Create your practice database

```
CREATE DATABASE accio_NN;
```

-- What just happened under the hood?

-- 1. PostgreSQL created a NEW directory on disk

-- 2. Copied the template database (template1) as the skeleton

-- 3. Created one default schema inside: 'public'

-- 4. Created system catalogs (pg_catalog, information_schema)

-- 5. The database has ZERO user tables. You build those.

-- Connect to it:

```
\c accio_NN      -- in psql
```

-- or: right-click in pgAdmin -> Query Tool

CREATE SCHEMA - Folders Inside the Database (1/4)

A schema is a namespace - a logical folder that groups related tables. Without schemas, all your tables dump into one flat 'public' schema. Imagine a folder with 55 files and no subfolders - chaos.

[Syntax & Examples]

```
CREATE SCHEMA schema_name;
```

```
-- Organize like a real company
```

```
CREATE SCHEMA shop;           -- products, inventory, suppliers  
CREATE SCHEMA people;        -- customers, employees  
CREATE SCHEMA transactions;   -- orders, payments, returns
```

CREATE SCHEMA - Folders Inside the Database (2/4)

```
-- Why schemas matter:  
-- 1. Organization: many tables in a few folders vs all loose  
-- 2. Namespace:    shop.products is different from analytics.products  
-- 3. Clarity:       SELECT * FROM sales.orders - you KNOW where it lives
```

CREATE SCHEMA - Folders Inside the Database (3/4)

[RetailMart V3's 16 Schemas - Why Each Exists]

Schema	Tables	Purpose
core	6	Static dimensions (dates, regions, categories)
products	4	What we sell (items, suppliers, inventory)
stores	3	Where we sell (locations, staff, expenses)
customers	5	Who buys (profiles, addresses, reviews, wallets)
sales	5	The transactions (orders, items, payments, shipments)
finance	6	Money tracking (accounts, transfers, revenue)
hr	2	People ops (attendance, salary history)
marketing	3	Campaign performance (ads, email clicks)
support	1	Customer complaints (tickets)
web_events	2	Digital tracking (page views, events)
supply_chain	3	Logistics (warehouses, shipments, stock)
loyalty	3	Rewards program (tiers, members, redemptions)
manufacture	2	Production (lines, work orders)
payroll	2	Salary processing (tax brackets, pay slips)
call_center	2	Phone support (calls, transcripts)
audit	6	System logs (API requests, changes, errors)

CREATE SCHEMA - Folders Inside the Database (4/4)

Total: 16 schemas, 55 tables. Every table has a HOME.

CREATE TABLE - The Main Event (1/5)

This is the DDL command you will use most. It defines a table - its name, its columns, and the data type of each column. Every column **MUST** have a type. Today, that type is always TEXT.

[Syntax - The Template]

```
CREATE TABLE schema_name.table_name (  
    column_1 TEXT,  
    column_2 TEXT,  
    column_3 TEXT,  
    ...  
);
```

```
-- If you skip schema_name, the table goes into the 'public' schema.  
-- ALWAYS specify the schema explicitly. It is a professional habit.
```

CREATE TABLE - The Main Event (2/5)

[Example 1 - A Students Table (all TEXT)]

```
CREATE TABLE public.students (  
    student_id    TEXT,  
    first_name    TEXT,  
    last_name     TEXT,  
    email         TEXT,  
    date_of_birth TEXT,  
    is_active     TEXT  
);
```

CREATE TABLE - The Main Event (3/5)

```
-- What happened?  
-- PostgreSQL created a table with 6 columns, all TEXT.  
-- Each column has a NAME and a TYPE (TEXT for now).  
-- The table has 0 rows. It is an empty blueprint waiting for data.  
-- Tomorrow: student_id -> INT, date_of_birth -> DATE, is_active -> BOOLEAN.
```

CREATE TABLE - The Main Event (4/5)

[Example 2 - A Products Table (inspired by RetailMart)]

```
CREATE TABLE shop.products (  
  product_id    TEXT,      -- tomorrow: INT (auto-ID with SERIAL)  
  product_name  TEXT,  
  brand_name    TEXT,  
  price         TEXT,      -- tomorrow: NUMERIC(10,2) for money  
  cost_price    TEXT,      -- tomorrow: NUMERIC(10,2)  
  category      TEXT,  
  is_available  TEXT      -- tomorrow: BOOLEAN  
);
```

```
-- Notice every column is TEXT. That is intentional.  
-- We are practising STRUCTURE today, not type selection.
```


CREATE TABLE - The Main Event (5/5)

[Example 3 - An Orders Table (core of any e-commerce DB)]

```
CREATE TABLE transactions.orders (  
  order_id      TEXT,  
  customer_id   TEXT,    -- will LINK to customers tomorrow (FOREIGN KEY)  
  store_id      TEXT,  
  order_date    TEXT,    -- tomorrow: DATE  
  order_status  TEXT,  
  gross_total   TEXT,    -- tomorrow: NUMERIC(12,2)  
  net_total     TEXT,  
  payment_mode  TEXT  
);
```

-- customer_id and store_id will REFERENCE other tables.

-- We make those links real (FOREIGN KEY) tomorrow. For now: plain TEXT.

Naming Conventions - The Unwritten Rules

[Professional Naming Standards]

RULE	Good	Bad
Use snake_case	first_name	firstName, FirstName
Use lowercase only	order_date	Order_Date, ORDER_DATE
Table = plural noun	customers, products	customer, product_tbl
Column = singular	city, order_status	cities, statuses
ID column = table_id	customer_id, product_id	id, cust, pid
Boolean = is_ / has_ prefix	is_active, has_projector	active, projector
Timestamp = _at suffix	created_at, updated_at	creation, update_time
Date = _date or _on suffix	order_date, enrolled_on	date, when
No reserved words	order_status	order, status, type

```
-- Why snake_case?  
-- PostgreSQL internally lowercases everything.  
-- 'OrderDate' becomes 'orderdate' unless you quote it. 'order_date' stays as-is.
```

```
-- RetailMart V3 follows these conventions exactly:  
-- customer_id, first_name, order_date, is_default.
```

CREATE TABLE IF NOT EXISTS - The Safety Net

[Why You Need This]

-- Without IF NOT EXISTS:

```
CREATE TABLE shop.products (product_id TEXT, product_name TEXT);
```

-- Run it once -> SUCCESS

-- Run it again -> ERROR: relation 'shop.products' already exists

-- With IF NOT EXISTS:

```
CREATE TABLE IF NOT EXISTS shop.products (  
    product_id    TEXT,  
    product_name TEXT  
);
```

-- Run it once -> SUCCESS

-- Run it again -> NOTICE: relation already exists, skipping

-- No error. Safe to run 100 times.

-- ALWAYS use this in scripts that might be run repeatedly:

-- setup scripts, migration scripts, CI/CD pipelines need idempotency.

DROP - The Demolition Crew

DROP permanently destroys a database object. There is no undo. No recycle bin. No Ctrl+Z. Gone forever.

[The DROP Commands]

-- Drop a table

```
DROP TABLE shop.products;
```

-- Drop a schema (fails if schema still has tables)

```
DROP SCHEMA shop;
```

-- ERROR: cannot drop schema shop because other objects depend on it

-- Drop schema AND everything inside it

```
DROP SCHEMA shop CASCADE;
```

-- CASCADE = 'destroy the folder AND all tables inside'. DANGEROUS.

-- Drop a database (must be connected to a DIFFERENT database)

```
DROP DATABASE accio_NN;
```

-- Safe versions (no error if it does not exist)

```
DROP TABLE IF EXISTS shop.products;
```

```
DROP SCHEMA IF EXISTS shop CASCADE;
```

DDL vs DML - The Line That Confuses Everyone

Question Students Ask	Answer
'DROP removes data, right?'	DROP removes the TABLE ITSELF + data. DELETE removes ROWS but the table stays.
'TRUNCATE is DDL or DML?'	TRUNCATE is DDL. It removes ALL rows but keeps the table structure intact.
DROP TABLE	-> Table is GONE. Columns gone. Data gone. Everything gone.
TRUNCATE	-> Table EXISTS but is EMPTY. Structure intact. 0 rows.
DELETE FROM	-> Table exists. Some rows removed. Others remain.
DROP	= Demolish the room
TRUNCATE	= Remove ALL furniture from the room (room stays)
DELETE	= Remove SPECIFIC furniture (room + other furniture stays)

PRACTICE BLOCK 1 - DDL: Building Structures

All practice in YOUR database (accio_NN). Every column is TEXT today - focus on structure. Do NOT touch RetailMart.

SCENARIO: You are the new Database Administrator at Prestige Engineering College. The dean wants to digitize all paper records. Start by building the structure from scratch.

YOUR TASK: Run the following DDL in `accio_NN`:

a) : Create a schema called college.

b) : Create college.students with columns: student_id, first_name, last_name, email, date_of_birth, is_active. All columns TEXT.

c) : Create college.courses with: course_id, course_name, credits, department. All TEXT.

d) : Verify both tables exist: `SELECT table_schema, table_name FROM information_schema.tables WHERE table_schema = 'college';`

e) : Inspect the columns: `\d college.students` (in psql) or pgAdmin's Columns tab.

Hint: Use `CREATE TABLE IF NOT EXISTS` for safety. Mistake? `DROP TABLE IF EXISTS` and recreate. That is the point of a practice database.

Answer



ANSWER

```
CREATE SCHEMA IF NOT EXISTS college;
```

```
CREATE TABLE IF NOT EXISTS college.students (  
    student_id    TEXT,  
    first_name    TEXT,  
    last_name     TEXT,  
    email         TEXT,  
    date_of_birth TEXT,  
    is_active     TEXT  
);
```

```
CREATE TABLE IF NOT EXISTS college.courses (  
    course_id    TEXT,  
    course_name  TEXT,  
    credits      TEXT,  
    department   TEXT  
);
```

```
SELECT table_schema, table_name  
FROM information_schema.tables  
WHERE table_schema = 'college';
```


SCENARIO: The college system is growing. The dean now wants to track teachers, classrooms, and fee payments. Design the tables a real college would need.

YOUR TASK:

a) : Create college.teachers with: teacher_id, first_name, last_name, email, phone, subject_specialty, monthly_salary, joining_date, is_hod. All TEXT.

b) : Create college.classrooms with: room_id, building_name, room_number, capacity, has_projector, has_ac. All TEXT.

c) : Create college.fee_payments with: payment_id, student_id, fee_type, amount, payment_date, payment_mode, academic_year, semester. All TEXT.

d) : List all tables in the college schema. How many do you have now?

Hint: Right now monthly_salary, amount, capacity etc. are all TEXT. Tomorrow you will ALTER them to NUMERIC / INT once real data shows what they need to be.

Answer (1/2)

✓ ANSWER

```
CREATE TABLE IF NOT EXISTS college.teachers (  
    teacher_id      TEXT,  
    first_name      TEXT,  
    last_name       TEXT,  
    email           TEXT,  
    phone           TEXT,  
    subject_specialty TEXT,  
    monthly_salary  TEXT,  
    joining_date    TEXT,  
    is_hod          TEXT  
);
```

```
CREATE TABLE IF NOT EXISTS college.classrooms (  
    room_id      TEXT,  
    building_name TEXT,  
    room_number  TEXT,  
    capacity     TEXT,  
    has_projector TEXT,  
    has_ac       TEXT  
);
```

```
CREATE TABLE IF NOT EXISTS college.fee_payments (  
    fee_id      TEXT,  
    student_id  TEXT,  
    amount      TEXT,  
    payment_date TEXT,  
    is_paid     TEXT  
);
```

Answer (2/2)

✓ ANSWER

```
payment_id    TEXT,  
student_id    TEXT,  
fee_type      TEXT,  
amount        TEXT,  
payment_date  TEXT,  
payment_mode  TEXT,  
academic_year TEXT,  
semester      TEXT  
);  
  
SELECT count(*) FROM information_schema.tables  
WHERE table_schema = 'college'; -- 5 tables now
```

SCENARIO: The Operations Manager at RetailMart says: 'Our supply chain team needs a new tracking system. Build me the schema in YOUR practice database.' You get only the business requirement - no column list.

YOUR TASK:

- a)** : Create a schema called supply in accio_NN.
- b)** : Design supply.warehouses - think: id, name, city, capacity, manager_name, is_active, established_date. All TEXT.
- c)** : Design supply.suppliers - company_name, contact_person, email, phone, city, state, registration_date. All TEXT.
- d)** : Design supply.shipments - shipment_id, supplier_id, warehouse_id, product_count, shipped_date, expected_arrival_date, actual_arrival_date, status. All TEXT.
- e)** : Run \d supply.shipments. Then compare to RetailMart's supply_chain.shipments (\d supply_chain.shipments). What did you miss? What did you add?

Hint: There are no wrong answers in STRUCTURE - only trade-offs. The point is to decide which COLUMNS each table needs. Types come tomorrow.

Answer (1/2)

✓ ANSWER

```
CREATE SCHEMA IF NOT EXISTS supply;
```

```
CREATE TABLE IF NOT EXISTS supply.warehouses (  
    warehouse_id    TEXT,  
    name            TEXT,  
    city            TEXT,  
    capacity        TEXT,  
    manager_name    TEXT,  
    is_active       TEXT,  
    established_date TEXT  
);
```

```
CREATE TABLE IF NOT EXISTS supply.suppliers (  
    supplier_id    TEXT,  
    company_name    TEXT,  
    contact_person  TEXT,  
    email           TEXT,  
    phone           TEXT,  
    city            TEXT,  
    state           TEXT,  
    registration_date TEXT  
);
```

Answer (2/2)

✓ ANSWER

```
CREATE TABLE IF NOT EXISTS supply.shipments (  
  shipment_id      TEXT,  
  supplier_id      TEXT,    -- will link -> suppliers tomorrow  
  warehouse_id     TEXT,    -- will link -> warehouses tomorrow  
  product_count    TEXT,  
  shipped_date      TEXT,  
  expected_arrival_date TEXT,  
  actual_arrival_date TEXT,  
  status           TEXT  
);
```

SCENARIO: The CTO says: 'We are prototyping RetailMart V4. Recreate the sales schema from scratch - just by looking at the current V3 design. Prove you understand DDL and naming.'

YOUR TASK:

- a)** : Create a schema called mini_mart in accio_NN.
- b)** : Look at RetailMart's sales.orders (\d sales.orders). Recreate it as mini_mart.orders with the SAME column names. All columns TEXT.
- c)** : Do the same for sales.order_items -> mini_mart.order_items.
- d)** : Do the same for customers.customers -> mini_mart.customers.
- e)** : List ALL columns of your mini_mart schema with the introspection query below, then run it for RetailMart's 'sales' schema and compare.

Hint: Match the real COLUMN NAMES exactly (cust_id, prod_id, net_total...). Every column is TEXT in your copy - tomorrow you will ALTER them to match the real types.

Introspection Query

✓ ANSWER

```
SELECT table_name, column_name, data_type
FROM information_schema.columns
WHERE table_schema = 'mini_mart'
ORDER BY table_name, ordinal_position;
```


Answer (1/2)

✓ ANSWER

```
CREATE SCHEMA IF NOT EXISTS mini_mart;
```

```
CREATE TABLE IF NOT EXISTS mini_mart.orders (  
  order_id      TEXT,  
  cust_id       TEXT,  
  store_id      TEXT,  
  order_date    TEXT,  
  order_status  TEXT,  
  gross_total   TEXT,  
  discount_amount TEXT,  
  net_total     TEXT,  
  payment_mode_id TEXT  
);
```

```
CREATE TABLE IF NOT EXISTS mini_mart.order_items (  
  order_item_id TEXT,  
  order_id      TEXT,  
  prod_id       TEXT,  
  quantity      TEXT,  
  unit_price     TEXT,  
  gross_amount  TEXT,  
  discount_amount TEXT,
```

Answer (2/2)



ANSWER

```
net_amount      TEXT
);

CREATE TABLE IF NOT EXISTS mini_mart.customers (
  customer_id    TEXT,
  first_name     TEXT,
  last_name      TEXT,
  email          TEXT,
  phone          TEXT,
  registration_date TEXT,
  tier            TEXT,
  tier_updated_at TEXT
);

SELECT table_name, column_name, data_type
FROM information_schema.columns
WHERE table_schema = 'mini_mart'
ORDER BY table_name, ordinal_position;
```

The Empty College Analogy

You just built the college schema - students, courses, teachers, fee_payments. Beautiful structure. But there is not a single student in it. No courses listed. The system is structurally perfect and completely useless.

DML is where you START USING your tables: INSERT adds rows, UPDATE changes them, DELETE removes them, TRUNCATE empties a table. Every row in every database on Earth got there because someone ran an INSERT.

Reminder: your columns are all TEXT today, so you type EVERY value as text - including ids ('1', '2', '3'). Tomorrow, when columns get proper types and auto-IDs (SERIAL), the database fills ids for you.

INSERT INTO - Adding Data (1/2)

Technical: INSERT INTO adds one or more new rows. You name the columns, then supply matching VALUES.

Layman's: INSERT is like filling out a new row in Excel - you type the values for each column.

[SYNTAX: INSERT INTO]

-- Single row (name the columns, then the values)

```
INSERT INTO schema.table (col1, col2, col3)
```

```
VALUES ('value1', 'value2', 'value3');
```

-- Multiple rows in one statement

```
INSERT INTO schema.table (col1, col2)
```

```
VALUES ('a', '1'), ('b', '2'), ('c', '3');
```

-- INSERT ... RETURNING (see what was inserted)

```
INSERT INTO schema.table (col1, col2)
```

```
VALUES ('value1', '42')
```

```
RETURNING col1, col2;
```

INSERT INTO - Adding Data (2/2)

-- Every value is in single quotes today - all columns are TEXT.

[Worked Example - Fill college.students]

```
INSERT INTO college.students
(student_id, first_name, last_name, email, date_of_birth, is_active)
VALUES ('1', 'Aarav', 'Sharma', 'aarav@college.edu', '2004-03-15', 'true');
```

-- Multi-row: add two more students at once

```
INSERT INTO college.students
(student_id, first_name, last_name, email, date_of_birth, is_active)
VALUES
('2', 'Diya', 'Patel', 'diya@college.edu', '2003-11-02', 'true'),
('3', 'Kabir', 'Singh', 'kabir@college.edu', '2004-07-21', 'false');
```

-- See the row you just added

```
INSERT INTO college.students
(student_id, first_name, last_name, email, date_of_birth, is_active)
VALUES ('4', 'Meera', 'Iyer', 'meera@college.edu', '2003-05-09', 'true')
RETURNING student_id, first_name, email;
```

UPDATE - Changing Existing Data (1/2)

Technical: UPDATE modifies existing rows. SET gives the new value(s); WHERE chooses which rows change. NO WHERE = EVERY row changes.

Layman's: UPDATE is 'Find and Replace' for your table - but if you forget WHERE, it replaces in EVERY row.

UPDATE - Changing Existing Data (2/2)

[SYNTAX & Worked Example]

-- Basic UPDATE with WHERE

```
UPDATE college.students  
SET email = 'aarav.sharma@college.edu'  
WHERE student_id = '1';
```

-- Change several columns at once

```
UPDATE college.students  
SET is_active = 'false', last_name = 'Sharma-Verma'  
WHERE student_id = '3';
```

-- RETURNING shows what changed

```
UPDATE college.students  
SET is_active = 'true'  
WHERE student_id = '3'  
RETURNING student_id, first_name, is_active;
```

Feeling the Pain of All-TEXT (and why Day 4 fixes it)

Because every column is TEXT today, two things feel awkward - on purpose:

- Math is clumsy. To raise a fee by 10% you cannot write `amount * 1.1` - `amount` is TEXT. You would have to cast: `amount = (amount::numeric * 1.1)::text`. Ugh.
- Comparisons sort like words, not numbers. `WHERE amount > '900'` compares TEXT, so '1000' is treated as LESS than '900' (it compares '1' vs '9'). Surprising and wrong for numbers.

That discomfort is the whole motivation for tomorrow: Data Types. You will ALTER `amount` to NUMERIC, date columns to DATE, `is_active` to BOOLEAN - and suddenly math and comparisons just work.

DELETE - Removing Data

Technical: DELETE FROM removes rows. WHERE chooses which rows. NO WHERE = ALL rows gone (the table structure stays).

Layman's: DELETE removes specific rows from a spreadsheet. The sheet and its headers remain. Forget WHERE and every row vanishes - but the empty sheet stays.

[SYNTAX & Worked Example]

-- Delete specific rows

```
DELETE FROM college.students WHERE student_id = '4';
```

-- Preview FIRST - what will this delete?

```
SELECT * FROM college.students WHERE is_active = 'false';
```

-- Happy with the preview? Delete with RETURNING to see what went

```
DELETE FROM college.students  
WHERE is_active = 'false'  
RETURNING student_id, first_name;
```

-- Delete ALL rows (table stays, structure intact) - dangerous!

```
-- DELETE FROM college.students;
```

TRUNCATE - The Nuclear Option

Technical: TRUNCATE removes ALL rows without scanning them one by one - far faster than DELETE for big tables. Structure stays. It is DDL, but in PostgreSQL it can be rolled back inside a transaction.

Layman's: DELETE erases each answer on an exam sheet one by one. TRUNCATE bins the sheet and hands you a fresh blank. Both give a blank sheet; TRUNCATE is instant.

[SYNTAX & Worked Example]

```
-- Empty a table fast (structure stays)
SELECT count(*) FROM college.students;    -- e.g. 3
TRUNCATE TABLE college.students;
SELECT count(*) FROM college.students;    -- 0

-- Table still works - insert fresh rows
INSERT INTO college.students (student_id, first_name)
VALUES ('1', 'Fresh Start');
```

-- Note: RESTART IDENTITY resets a SERIAL counter to 1.
-- Your columns are TEXT today (no SERIAL), so you will use that tomorrow.

DROP vs TRUNCATE vs DELETE

Q: 'What is the difference between DROP, TRUNCATE, and DELETE?'

A: 'DROP removes the table itself - structure and data gone, the table ceases to exist. TRUNCATE removes ALL rows but keeps the structure - fast, no WHERE. DELETE removes specific rows via a WHERE condition - precise, and you can roll it back inside a transaction. DROP and TRUNCATE are DDL; DELETE is DML. Memory hook: DROP = demolish the room, TRUNCATE = empty the room, DELETE = remove specific furniture.'

PRACTICE BLOCK 2 - DML: Putting Data In

Still in accio_NN, still all-TEXT columns. Build on the tables from Block 1. Do NOT touch RetailMart.

SCENARIO: Admissions are open at Prestige College. Time to put your first students and courses into the tables you built.

YOUR TASK:

a) : INSERT one student into college.students (all 6 columns) - student_id '1'.

b) : INSERT three more students in a SINGLE multi-row INSERT (ids '2','3','4').

c) : INSERT two courses into college.courses.

d) : Use RETURNING on one INSERT to see what was added.

Hint: Every value is in single quotes - all columns are TEXT. Multi-row INSERT: VALUES (...), (...), (...).

Answer



ANSWER

```
INSERT INTO college.students
(student_id, first_name, last_name, email, date_of_birth, is_active)
VALUES ('1', 'Aarav', 'Sharma', 'aarav@college.edu', '2004-03-15', 'true');
```

```
INSERT INTO college.students
(student_id, first_name, last_name, email, date_of_birth, is_active)
VALUES
('2', 'Diya', 'Patel', 'diya@college.edu', '2003-11-02', 'true'),
('3', 'Kabir', 'Singh', 'kabir@college.edu', '2004-07-21', 'false'),
('4', 'Meera', 'Iyer', 'meera@college.edu', '2003-05-09', 'true');
```

```
INSERT INTO college.courses (course_id, course_name, credits, department)
VALUES ('1', 'Database Systems', '4', 'Computer Science')
RETURNING course_id, course_name;
```

```
INSERT INTO college.courses (course_id, course_name, credits, department)
VALUES ('2', 'Data Structures', '4', 'Computer Science');
```

SCENARIO: Records change. A student updates their email, one drops out, and the registrar wants to deactivate a specific student - carefully, by id.

YOUR TASK:

- a)** : UPDATE student_id '1' - change email to 'aarav.sharma@college.edu'.
- b)** : UPDATE student_id '3' - set is_active to 'false' AND last_name to 'Singh-Rao' in ONE statement.
- c)** : Preview with SELECT, then DELETE student_id '4', using RETURNING to confirm what went.
- d)** : Verify the remaining rows with a SELECT.

Hint: Always UPDATE/DELETE by a single id to affect exactly one row. SET col1 = .., col2 = .. updates multiple columns at once.

Answer



```
UPDATE college.students  
SET email = 'aarav.sharma@college.edu'  
WHERE student_id = '1';
```

```
UPDATE college.students  
SET is_active = 'false', last_name = 'Singh-Rao'  
WHERE student_id = '3';
```

```
-- Preview first  
SELECT * FROM college.students WHERE student_id = '4';
```

```
DELETE FROM college.students  
WHERE student_id = '4'  
RETURNING student_id, first_name;
```

```
SELECT student_id, first_name, is_active FROM college.students  
ORDER BY student_id;
```


SCENARIO: The fee desk opens. You record fee payments, fix a wrongly-entered amount, remove a duplicate, then a new semester starts and the fees table must be wiped for a fresh import.

YOUR TASK:

- a)** : INSERT 4 rows into college.fee_payments (mix of tuition/hostel/library).
- b)** : UPDATE one payment whose amount was typed wrong.
- c)** : DELETE the duplicate payment (payment_id '3') with RETURNING.
- d)** : New semester: TRUNCATE college.fee_payments, then prove it is empty with count(*) .

Hint: TRUNCATE clears the whole table fast and keeps the structure - perfect before a fresh bulk import. amount stays TEXT today; that is fine.

Answer



ANSWER

```
INSERT INTO college.fee_payments
(payment_id, student_id, fee_type, amount, payment_date,
 payment_mode, academic_year, semester)
VALUES
('1', '1', 'tuition', '85000', '2026-01-05', 'UPI', '2025-2026', '1'),
('2', '2', 'hostel', '45000', '2026-01-06', 'Card', '2025-2026', '1'),
('3', '2', 'hostel', '45000', '2026-01-06', 'Card', '2025-2026', '1'),
('4', '3', 'library', '2000', '2026-01-07', 'Cash', '2025-2026', '1');
```

-- (b) amount was wrong for payment 1

```
UPDATE college.fee_payments
SET amount = '90000'
WHERE payment_id = '1';
```

-- (c) row 3 is a duplicate of row 2

```
DELETE FROM college.fee_payments
WHERE payment_id = '3'
RETURNING payment_id, student_id, amount;
```

-- (d) new semester: wipe for fresh import

```
TRUNCATE TABLE college.fee_payments;
SELECT count(*) FROM college.fee_payments;    -- 0
```

SCENARIO: You prototype a tiny orders table from scratch, load it, run a full set of DML operations - and discover firsthand why all-TEXT columns make number work painful. This is the bridge to tomorrow.

YOUR TASK:

a) : Create schema shop and table shop.mini_orders (order_id, customer_name, status, total) - all TEXT.

b) : INSERT 5 orders (totals like '900', '1000', '250', '4999', '120').

c) : Mark order '1' as 'Delivered' (UPDATE).

d) : Run SELECT ... WHERE total > '900' and look at the result - notice '1000' is MISSING (TEXT compares like words!).

e) : DELETE all 'Cancelled' orders. Then TRUNCATE the table for a clean slate.

Hint: The WHERE total > '900' surprise is the lesson: TEXT sorts character-by-character, so '1000' < '900'. Tomorrow you ALTER total to NUMERIC and the comparison behaves like real numbers.

Answer (1/2)

✓ ANSWER

```
CREATE SCHEMA IF NOT EXISTS shop;
```

```
CREATE TABLE IF NOT EXISTS shop.mini_orders (  
    order_id      TEXT,  
    customer_name TEXT,  
    status        TEXT,  
    total         TEXT  
);
```

```
INSERT INTO shop.mini_orders (order_id, customer_name, status, total)  
VALUES  
    ('1', 'Aarav', 'Pending', '900'),  
    ('2', 'Diya', 'Pending', '1000'),  
    ('3', 'Kabir', 'Cancelled', '250'),  
    ('4', 'Meera', 'Pending', '4999'),  
    ('5', 'Ravi', 'Cancelled', '120');
```

```
UPDATE shop.mini_orders SET status = 'Delivered' WHERE order_id = '1';
```

```
-- SURPRISE: '1000' is NOT returned - TEXT compares char-by-char,  
-- so '1000' < '900'. This is the motivation for Data Types (tomorrow).  
SELECT order_id, total FROM shop.mini_orders WHERE total > '900';
```

Answer (2/2)

✓ ANSWER

```
DELETE FROM shop.mini_orders WHERE status = 'Cancelled';  
TRUNCATE TABLE shop.mini_orders;
```

Key Takeaways (1/2)

- 1. DDL = Structure:** CREATE builds objects, DROP destroys them. CREATE DATABASE / SCHEMA / TABLE define the blueprint - what tables exist, what columns, organized into schemas.
- 2. Every column is TEXT today:** We focus on STRUCTURE, not type selection. CREATE TABLE is just 'column_name TEXT' for each column. Types come tomorrow.
- 3. Safety habits:** Always specify the schema. Use IF NOT EXISTS for idempotent scripts. DROP is permanent - practice only in accio_NN, never on RetailMart.

Key Takeaways (2/2)

4. DML = Data: INSERT adds rows (single, multi-row, RETURNING). UPDATE changes rows. DELETE removes rows. All take WHERE - and forgetting it hits EVERY row.

5. ALWAYS WHERE: UPDATE/DELETE without WHERE changes/removes every row. Preview with SELECT first, then run the change.

6. DROP vs TRUNCATE vs DELETE: DROP = demolish the room (structure gone). TRUNCATE = empty the room fast (structure stays). DELETE = remove specific furniture (WHERE).

7. The all-TEXT pain is the point: Math and comparisons are awkward on TEXT. That discomfort sets up tomorrow's Data Types lesson, where you ALTER columns to their proper types.

DDL & DML - Quick Reference

-- DDL Commands

```
CREATE DATABASE db_name;
CREATE SCHEMA schema_name;
CREATE SCHEMA IF NOT EXISTS schema_name;
CREATE TABLE schema.table (col TEXT, col TEXT, ...);
CREATE TABLE IF NOT EXISTS schema.table (...);
DROP TABLE schema.table;
DROP TABLE IF EXISTS schema.table;
DROP SCHEMA IF EXISTS schema CASCADE;
```

-- INSERT

```
INSERT INTO schema.table (c1, c2)
VALUES ('a', 'b');
-- multi-row:
INSERT INTO schema.table (c1, c2)
VALUES ('a','1'), ('b','2'), ('c','3');
-- see what was added:
INSERT INTO ... VALUES (...) RETURNING c1, c2;
```

-- UPDATE

```
UPDATE schema.table
SET col1 = 'new', col2 = 'new2'
WHERE id_col = '1';
-- ALWAYS include WHERE!
UPDATE ... SET ... WHERE ... RETURNING col;
```


DDL & DML - Quick Reference

-- DELETE / TRUNCATE

```
DELETE FROM schema.table WHERE id_col = '1';
DELETE FROM ... WHERE ... RETURNING col;
-- empty the whole table fast (structure stays):
TRUNCATE TABLE schema.table;
```

-- Inspect Your Work

```
\d schema.table          -- columns (psql)
SELECT * FROM schema.table;
SELECT count(*) FROM schema.table;
SELECT table_name, column_name, data_type
FROM information_schema.columns
WHERE table_schema = 'your_schema';
```

-- Golden Rules

```
Every column needs a TYPE (TEXT today)
Always name the schema: schema.table
IF NOT EXISTS / IF EXISTS for safe re-runs
UPDATE & DELETE: never forget WHERE
Preview with SELECT before UPDATE/DELETE
Never change RetailMart - build in accio_NN
```

INTERVIEW QUESTION BANK - DDL & DML

Practice answering OUT LOUD. Interviews test how clearly you explain, not just whether you know.

Interview Questions - DDL & DML

Q1: 'What is DDL? Give 3 examples.'

A: 'DDL is Data Definition Language - commands that define or change database STRUCTURE. CREATE TABLE builds a table, ALTER TABLE modifies one, DROP TABLE destroys one. DDL changes the blueprint, not the data inside.'

Q2: 'What is DML? Give 3 examples.'

A: 'DML is Data Manipulation Language - commands that work with the DATA in tables. INSERT adds rows, UPDATE changes rows, DELETE removes rows. The structure stays the same; only the rows change.'

Q3: 'Difference between DROP TABLE and DELETE FROM?'

A: 'DROP is DDL - it destroys the whole table, structure and data; the table no longer exists. DELETE is DML - it removes specific rows but the table stays, and you can INSERT into it again afterwards.'

Q4: 'Difference between DROP, TRUNCATE, and DELETE?'

A: 'DROP destroys the table entirely. TRUNCATE removes ALL rows but keeps the structure - fast, no WHERE. DELETE removes specific rows via WHERE. DROP and TRUNCATE are DDL; DELETE is DML.'

Q5: 'What is a schema in PostgreSQL? Why not just use public?'

DDL & DML Self-Check (12 Questions)

Close your notes. Answer from memory:

- 1. What does DDL stand for? Name 3 DDL commands.
- 2. What does DML stand for? Name 3 DML commands.
- 3. Write the CREATE TABLE syntax from memory (3 TEXT columns).
- 4. Why is every column TEXT today? What changes tomorrow?
- 5. Difference between DROP TABLE, TRUNCATE, and DELETE?
- 6. What is a schema and why use one instead of public?
- 7. What does CREATE TABLE IF NOT EXISTS do?
- 8. Write a single-row INSERT and a multi-row INSERT.
- 9. Why is UPDATE without WHERE dangerous? How do you stay safe?
- 10. What does RETURNING give you?
- 11. Why does WHERE total > '900' miss the row with '1000'?
- 12. What is CASCADE in DROP SCHEMA?

Score 10+ -> ready for Data Types & Constraints. Below 10 -> re-read the weak sections.

DDL & DML Take-Home Challenge

Complete these BEFORE Data Types & Constraints:

- 1. Build 3 Tables:** : In `accio_NN`, create a mini e-commerce schema with products (7+ columns), customers (6+ columns), orders (7+ columns). ALL columns TEXT. Use `schema.table` names and IF NOT EXISTS.
- 2. Fill Them:** : INSERT at least 5 rows into each table - use a multi-row INSERT at least once, and RETURNING at least once.
- 3. Change Data:** : Write 3 UPDATE statements (each with WHERE) and 2 DELETE statements (each with WHERE). Preview each with a SELECT first.
- 4. Reset:** : TRUNCATE one table, then prove it is empty with `count(*)`.
- 5. Inspect:** : Run the `information_schema.columns` query on your schema and note that EVERY column shows `data_type = text`. Tomorrow you will fix that.

Tomorrow: Data Types (ALTER those TEXT columns to INT / NUMERIC / DATE / BOOLEAN) and Constraints & Keys (PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, CHECK).

YOUR SQL JOURNEY



SQL Intro



Installation



DDL + DML <- YOU ARE HERE

← HERE



Data Types + Constraints



Filtering (first DQL!)



Joins



CTEs



Window Functions



Indexing



Views



Cohort/RFM



Capstone



WhatsApp Announcement

Tomorrow you give your columns their PROPER types and add RULES that protect your data:

- Data Types - INT, NUMERIC, DATE, TIMESTAMP, BOOLEAN, and friends
- ALTER TABLE - convert today's TEXT columns into their proper types
- NOT NULL, DEFAULT, UNIQUE, CHECK - data integrity constraints
- PRIMARY KEY - uniquely identify every row
- FOREIGN KEY - link tables together (the 'relational' part!)

Today you built the structure and filled it with data. Tomorrow you make that data CORRECT by type and SAFE by rule.

- See you on Data Types, Constraints & Keys! Siraj Sir